



UNIVERSIDADE VEIGA DE ALMEIDA
CURSO DE CIÊNCIA DA COMPUTAÇÃO

ANNA CAROLINA MILITÃO DOS SANTOS (1240114001)
NATHÁLIA BARBOSA (1240102519)
JOÃO PEDRO SALAMA RANGEL (1240102519)
THIAGO ESPIRITO SANTO MIRÓ (1240105718)
VICTOR MAGNO COELHO (1240106929)
VINÍCIUS DA SILVA NOGUEIRA (1240109652)

**DESENVOLVIMENTO DE UM ANALISADOR LÉXICO PARA A LINGUAGEM
DOMUS**

Rio de Janeiro
2026

ANNA CAROLINA MILITÃO DOS SANTOS (1240114001)

NATHÁLIA BARBOSA (1240102519)

JOÃO PEDRO SALAMA RANGEL (1240102519)

THIAGO ESPIRITO SANTO MIRÓ (1240105718)

VICTOR MAGNO COELHO (1240106929)

VINÍCIUS DA SILVA NOGUEIRA (1240109652)

DESENVOLVIMENTO DE UM ANALISADOR LÉXICO PARA A LINGUAGEM DOMUS

Trabalho apresentado ao curso de Ciência da
Computação da Universidade Veiga de Almeida
para obtenção de nota na disciplina Compiladores.

Orientador: Fábio Contarini Carneiro

Rio de Janeiro

2026

RESUMO

Este trabalho apresenta o desenvolvimento de um analisador léxico para a linguagem Domus, utilizando a ferramenta Flex em conjunto com a linguagem C. O analisador foi construído para reconhecer palavras-chave, identificadores, números, operadores, separadores e símbolos desconhecidos que estão no código-fonte. Também foram implementados o controle de linhas e o tratamento de comentários, o que facilita a identificação dos elementos durante a análise. Para verificar se o programa estava funcionando da maneira correta, foram utilizados arquivos de teste contendo comandos válidos e símbolos não reconhecidos. Os testes permitiram observar o comportamento do analisador em diferentes situações e confirmar que os principais componentes léxicos da linguagem foram identificados corretamente. Além da implementação, o trabalho busca mostrar de forma clara como o Flex pode ser utilizado na construção de analisadores léxicos simples, relacionando as regras definidas no arquivo de entrada com os tokens apresentados na saída do programa e com os resultados obtidos durante a execução dos testes realizados.

Palavras-chave: análise léxica; Flex; linguagem Domus; compiladores; tokens.

SUMÁRIO

1 INTRODUÇÃO.....	5
2 LEXEMAS E TOKENS DA LINGUAGEM DOMUS.....	5
3.1 Estrutura do arquivo de entrada do Flex.....	7
3.2 Renomeação da função principal com YY_DECL.....	8
3.3 Tratamento de comentários com estados exclusivos.....	8
3.4 Ordem das regras e resolução de ambiguidades.....	8
3.5 A variável global yytext.....	9
4 UTILIZAÇÃO DO GERADOR AUTOMÁTICO (FLEX).....	9
5 COMPILAÇÃO E EXECUÇÃO.....	10
5.1 Compilação e execução no macOS / Linux.....	10
5.2 Compilação e execução no Windows.....	10
6 RESULTADOS DOS TESTES.....	10
6.1 Teste 1 – ExemploDomus.txt.....	11
Figura 1 – Saída do terminal para o Teste 1 (ExemploDomus.txt).....	12
6.2 Teste 2 – NossoExemplo.txt.....	13
Figura 2 – Saída do terminal para o Teste 2 (NossoExemplo.txt).....	13
6.3 Teste 3 – Reconhecimento de símbolos desconhecidos.....	14
Figura 3 – Saída do terminal para o Teste 3 (símbolos desconhecidos).....	15
7. CONSIDERAÇÕES FINAIS.....	16
REFERÊNCIAS.....	17

1 INTRODUÇÃO

Este documento apresenta a construção e a configuração passo a passo de um analisador léxico para a linguagem Domus.

Para a implementação, o grupo optou pela utilização do gerador automático de analisadores léxicos Flex, que a partir de um conjunto de expressões regulares gera um código-fonte em linguagem C responsável pelo reconhecimento dos tokens da linguagem-alvo.

2 LEXEMAS E TOKENS DA LINGUAGEM DOMUS

Um token representa a categoria sintática de uma unidade léxica (por exemplo, um identificador ou uma palavra-chave), enquanto o lexema é a sequência de caracteres real reconhecida no código-fonte que corresponde a esse token. O Quadro 1 apresenta, de forma consolidada, todos os lexemas e tokens definidos pelas convenções léxicas da linguagem Domus.

Categoria	Lexema	Token	Descrição
Palavra-chave	sensor	Palavra-chave	Definição de um sensor (variável de somente leitura)
Palavra-chave	atuador	Palavra-chave	Definição de um atuador (variável de somente escrita)
Palavra-chave	porta	Palavra-chave	Definição da porta onde o sensor/atuador está instalado
Palavra-chave	analogico	Palavra-chave	Define o dispositivo como analógico (valores inteiros)
Palavra-chave	digital	Palavra-chave	Define o dispositivo como digital (valores 0 ou 1)
Palavra-chave	se	Palavra-chave	Início da estrutura de decisão (equivalente ao if)
Palavra-chave	entao	Palavra-chave	Bloco executado caso a decisão seja verdadeira
Palavra-chave	senao	Palavra-chave	Bloco executado caso a decisão seja falsa
Palavra-chave	fim_se	Palavra-chave	Final da estrutura de decisão
Palavra-chave	enquanto	Palavra-chave	Estrutura de repetição (equivalente ao while)
Palavra-chave	fim_enquanto	Palavra-chave	Final da estrutura de repetição
Palavra-chave	ler	Palavra-chave	Rotina de entrada de dados
Palavra-chave	escrever	Palavra-chave	Rotina de saída de dados
Operador	+	Operação de soma	Soma aritmética
Operador	-	Operação de subtração	Subtração aritmética
Operador	*	Operação de multiplicação	Multiplicação aritmética
Operador	/	Operação de divisão	Divisão aritmética
Operador	<	Menor que	Comparação relacional
Operador	<=	Menor ou igual	Comparação relacional
Operador	>	Maior que	Comparação relacional
Operador	>=	Maior ou igual	Comparação relacional
Operador	==	Comparação igual	Comparação de igualdade
Operador	!=	Comparação diferente	Comparação de desigualdade
Operador	<-	Operador de atribuição	Atribuição de valor a uma variável
Separador	.	Separador de comandos	Marca o fim de um comando
Separador	:	Separador de tipo	Separa identificador do tipo do dispositivo
Separador	,	Separador de declarações	Separa itens em uma lista de declarações
Separador	(	Abre bloco	Abertura de bloco de expressão
Separador	)	Fecha bloco	Fechamento de bloco de expressão
Outro	[a-zA-Z][a-zA-Z0-9]*	Identificador	Sequência de letras e números iniciada por letra
Outro	[0-9]+	Número inteiro	Sequência de um ou mais dígitos
Outro	EOF	Fim de arquivo	Indica que o arquivo de entrada terminou

Além dos elementos apresentados no Quadro 1, espaços e tabulações não geram tokens durante a análise. As quebras de linha também não são classificadas como tokens, mas servem para atualizar a contagem das linhas do arquivo. Os comentários são delimitados por `/*` e `*/`, podem ocupar várias linhas e são descartados pelo analisador.

3. ESTRATÉGIA DE DESENVOLVIMENTO

O grupo optou por utilizar o Flex para a construção do analisador léxico. A ferramenta utiliza as expressões regulares que já foram definidas no arquivo de entrada para gerar, em linguagem C, o código que reconhece os lexemas da linguagem Domus. Dessa forma, o desenvolvimento fica concentrado principalmente na definição dos padrões léxicos e das ações associadas a cada regra.

3.1 Estrutura do arquivo de entrada do Flex

O arquivo-fonte do Flex (`analisador.l`) é dividido em três seções, separadas pelo delimitador `%%`:

```
{definições}
%%
{regras}
%%
{rotinas auxiliares}
```

Na primeira seção, são incluídas as definições utilizadas pelo analisador, como as bibliotecas de C, a variável `linha_atual` e a macro `YY_DECL`. Na segunda seção, são definidas as regras de reconhecimento dos lexemas, associando cada padrão a uma ação escrita em C. A última seção contém as funções auxiliares utilizadas pelo programa, como `yywrap()` e `main()`.

3.2 Renomeação da função principal com YY_DECL

O trabalho exige que a função de análise se chame `getToken()`. Como o Flex gera, por padrão, uma função chamada `yylex()`, foi utilizada a macro `YY_DECL` para definir que a função de análise se chamasse `getToken()` e tivesse retorno do tipo inteiro.

```
#define YY_DECL int getToken()
```

Após essa definição, a função gerada pelo Flex passa a ser utilizada com o nome `getToken()`, sem a necessidade da alteração manual do arquivo `lex.yy.c`.

3.3 Tratamento de comentários com estados exclusivos

Os comentários da linguagem Domus podem acabar ocupando várias linhas. Para tratá-los, foi utilizado o estado exclusivo `%x COMENTARIO` do Flex. Ao encontrar `/*`, o analisador entra no estado `COMENTARIO` por meio de `BEGIN(COMENTARIO)`. Durante esse estado, o conteúdo do comentário é ignorado, enquanto as quebras de linha continuam sendo contadas. Ao encontrar `*/`, o analisador retorna ao seu estado inicial com `BEGIN(INITIAL)`.

3.4 Ordem das regras e resolução de ambiguidades

Quando mais de uma regra pode reconhecer o mesmo trecho, o Flex dá prioridade para a correspondência que identifica a maior quantidade de caracteres. Em caso de empate, é utilizada a regra que aparece primeiro no arquivo.

Por esse motivo, as palavras-chave, como `sensor`, `se` e `enquanto`, foram definidas antes da regra geral de identificadores: `([a-zA-Z_][a-zA-Z0-9_]*)`. Essa ordem garante que palavras reservadas sejam classificadas do jeito correto.

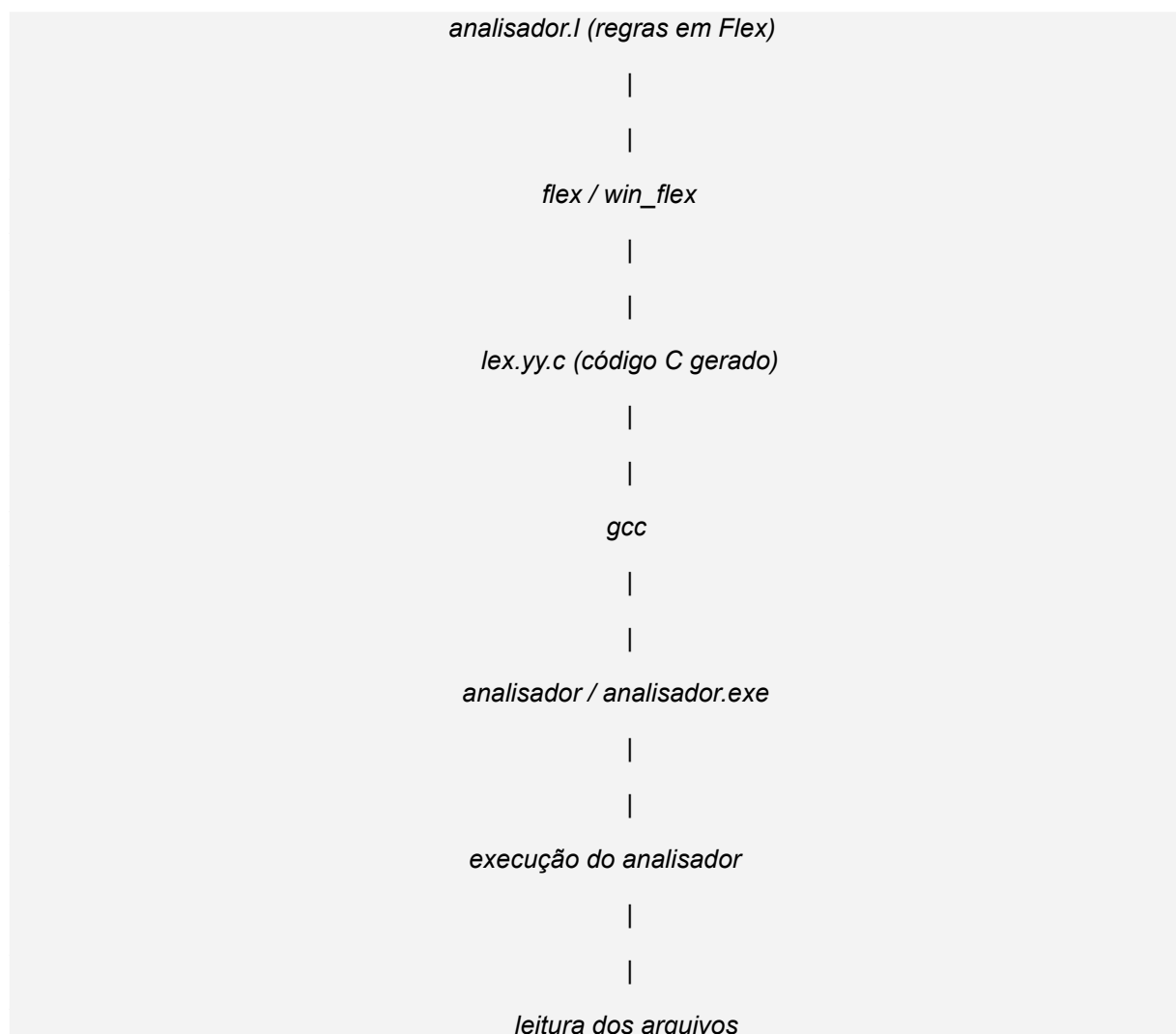
3.5 A variável global yytext

A variável `yytext`, que é fornecida pelo próprio Flex, armazena o texto reconhecido pela regra atual. No analisador que foi desenvolvido, ela é utilizada para exibir o valor de cada token identificado enquanto o arquivo está sendo lido.

4 UTILIZAÇÃO DO GERADOR AUTOMÁTICO (FLEX)

As regras do analisador são adequadamente definidas no arquivo `analisador.l`. A partir delas, o Flex gera o código em C responsável pela análise léxica, armazenado no arquivo `lex.yy.c`. Depois disso, o código é compilado com o GCC para criar o programa que será posteriormente executado nos testes.

As etapas de geração, compilação e execução do analisador que foram utilizadas pelo grupo são apresentadas a seguir:



5 COMPILAÇÃO E EXECUÇÃO

O grupo desenvolveu e testou o analisador em dois ambientes diferentes: macOS, utilizando Flex e GCC, e Windows, utilizando WinFlexBison e MinGW-w64. Em ambos os ambientes, o processo segue a mesma lógica: o Flex gera o código em C e, logo em seguida, o GCC realiza a compilação.

5.1 Compilação e execução no macOS / Linux

```
flex analisador.l  
gcc lex.yy.c -o analisador  
./analisador
```

5.2 Compilação e execução no Windows

No Windows, foram utilizadas as ferramentas WinFlexBison, um port do Flex e do Bison para Windows, e MinGW-w64, um conjunto de ferramentas que inclui o GCC. Ambas foram adicionadas à variável de ambiente Path do sistema.

```
win_flex analisador.l  
gcc lex.yy.c -o analisador.exe  
analisador.exe
```

Como o programa faz uma procura pelo arquivo ExemploDomus.txt, esse arquivo deve estar disponível no diretório em que o analisador for executado, mantendo o mesmo nome definido na função `fopen()`.

6 RESULTADOS DOS TESTES

Para verificar o funcionamento do analisador léxico, foram realizados três testes distintos: (i) com o arquivo oficial disponibilizado pelo professor; (ii) com um arquivo elaborado pelo próprio grupo; e (iii) com um arquivo contendo símbolos não pertencentes à linguagem Domus, a fim de observar o tratamento de erros léxicos.

6.1 Teste 1 – ExemploDomus.txt

O primeiro teste foi feito em cima do arquivo ExemploDomus.txt, disponibilizado no Teams como arquivo de teste do trabalho. O código-fonte utilizado é apresentado a seguir.

```
/* Programa de teste */
sensor temperatura: analogico(porta 1).
sensor claridade:digital(porta 2).
/* multiplas
linhas
de comentario
*/
atuador cortina: digital (porta 3).
atuador ventilador:analogico(porta 4).

se (((temperatura < 30))) entao
    cortina <- 1.
    ventiladora <- 80.
senao
    cortina <- 0.
    ventilador <- 0.
fim_se .
asd <- 8 .
asd <- asd + 2 .
escrever(asd+2) .
enquanto (asd>1)
    asd <- asd -1.
fim_enquanto.
```

A Figura 1 apresenta qual foi a saída produzida pelo analisador léxico após o processamento do arquivo, exibindo corretamente todos os tokens, seus respectivos lexemas e o número da linha onde cada um foi encontrado, incluindo o tratamento do comentário de múltiplas linhas presente entre as linhas 4 e 7.

Figura 1 – Saída do terminal para o Teste 1 (ExemploDomus.txt)

```

• viniciusnogueira@MacBook-Air-de-Vinicius AnalisadorLexico % flex analisador.l
gcc lex.yy.c -o analisador
./analisador
Início do Analisador Léxico.

Token: Palavra-chave      Valor: sensor      Linha: 2
Token: Identificador      Valor: temperatura Linha: 2
Token: Separador de tipo  Valor: :           Linha: 2
Token: Palavra-chave      Valor: analogico   Linha: 2
Token: Abre bloco         Valor: (           Linha: 2
Token: Palavra-chave      Valor: porta       Linha: 2
Token: Numero inteiro     Valor: 1           Linha: 2
Token: Fecha bloco        Valor: )           Linha: 2
Token: Separador de comandos Valor: .           Linha: 2
Token: Palavra-chave      Valor: sensor      Linha: 3
Token: Identificador      Valor: claridade   Linha: 3
Token: Separador de tipo  Valor: :           Linha: 3
Token: Palavra-chave      Valor: digital     Linha: 3
Token: Abre bloco         Valor: (           Linha: 3
Token: Palavra-chave      Valor: porta       Linha: 3
Token: Numero inteiro     Valor: 2           Linha: 3
Token: Fecha bloco        Valor: )           Linha: 3
Token: Separador de comandos Valor: .           Linha: 3
Token: Palavra-chave      Valor: atuador     Linha: 8
Token: Identificador      Valor: cortina     Linha: 8
Token: Separador de tipo  Valor: :           Linha: 8
Token: Palavra-chave      Valor: digital     Linha: 8
Token: Abre bloco         Valor: (           Linha: 8
Token: Palavra-chave      Valor: porta       Linha: 8
Token: Numero inteiro     Valor: 3           Linha: 8
Token: Fecha bloco        Valor: )           Linha: 8
Token: Separador de comandos Valor: .           Linha: 8
Token: Palavra-chave      Valor: atuador     Linha: 9
Token: Identificador      Valor: ventilador  Linha: 9
Token: Separador de tipo  Valor: :           Linha: 9
Token: Palavra-chave      Valor: analogico   Linha: 9
Token: Abre bloco         Valor: (           Linha: 9
Token: Palavra-chave      Valor: porta       Linha: 9
Token: Numero inteiro     Valor: 4           Linha: 9
Token: Fecha bloco        Valor: )           Linha: 9
Token: Separador de comandos Valor: .           Linha: 9
Token: Palavra-chave      Valor: se          Linha: 11
Token: Abre bloco         Valor: (           Linha: 11
Token: Abre bloco         Valor: (           Linha: 11
Token: Abre bloco         Valor: (           Linha: 11
Token: Identificador      Valor: temperatura Linha: 11
Token: Menor que          Valor: <           Linha: 11
Token: Numero inteiro     Valor: 30          Linha: 11
Token: Fecha bloco        Valor: )           Linha: 11
Token: Fecha bloco        Valor: )           Linha: 11
Token: Fecha bloco        Valor: )           Linha: 11
Token: Palavra-chave      Valor: entao       Linha: 11
Token: Identificador      Valor: cortina     Linha: 12
Token: Operador de atribuicao Valor: <-          Linha: 12
Token: Numero inteiro     Valor: 1           Linha: 12
Token: Separador de comandos Valor: .           Linha: 12
Token: Identificador      Valor: ventiladora Linha: 13
Token: Operador de atribuicao Valor: <-          Linha: 13
Token: Numero inteiro     Valor: 80          Linha: 13
Token: Separador de comandos Valor: .           Linha: 13
Token: Palavra-chave      Valor: senao       Linha: 14
Token: Identificador      Valor: cortina     Linha: 15
Token: Operador de atribuicao Valor: <-          Linha: 15
Token: Numero inteiro     Valor: 0           Linha: 15
Token: Separador de comandos Valor: .           Linha: 15
Token: Identificador      Valor: ventilador  Linha: 16
Token: Operador de atribuicao Valor: <-          Linha: 16
Token: Numero inteiro     Valor: 0           Linha: 16
Token: Separador de comandos Valor: .           Linha: 16
Token: Palavra-chave      Valor: fim_se      Linha: 17
Token: Separador de comandos Valor: .           Linha: 17
Token: Identificador      Valor: asd         Linha: 18
Token: Operador de atribuicao Valor: <-          Linha: 18
Token: Numero inteiro     Valor: 8           Linha: 18
Token: Separador de comandos Valor: .           Linha: 18
Token: Identificador      Valor: asd         Linha: 19
Token: Operador de atribuicao Valor: <-          Linha: 19
Token: Identificador      Valor: asd         Linha: 19
Token: Operacao de soma   Valor: +           Linha: 19
Token: Numero inteiro     Valor: 2           Linha: 19
Token: Separador de comandos Valor: .           Linha: 19
Token: Palavra-chave      Valor: escrever    Linha: 20
Token: Abre bloco         Valor: (           Linha: 20
Token: Identificador      Valor: asd         Linha: 20
Token: Operacao de soma   Valor: +           Linha: 20
Token: Numero inteiro     Valor: 2           Linha: 20
Token: Fecha bloco        Valor: )           Linha: 20
Token: Separador de comandos Valor: .           Linha: 20
Token: Palavra-chave      Valor: enquanto     Linha: 21
Token: Abre bloco         Valor: (           Linha: 21
Token: Identificador      Valor: asd         Linha: 21
Token: Maior que          Valor: >           Linha: 21
Token: Numero inteiro     Valor: 1           Linha: 21
Token: Fecha bloco        Valor: )           Linha: 21

Token: Operador de atribuicao Valor: <-          Linha: 22
Token: Identificador      Valor: asd         Linha: 22
Token: Operacao de subtracao Valor: -           Linha: 22
Token: Numero inteiro     Valor: 1           Linha: 22
Token: Separador de comandos Valor: .           Linha: 22
Token: Palavra-chave      Valor: fim_enquanto Linha: 23
Token: Separador de comandos Valor: .           Linha: 23
Token: Fim de arquivo     Valor: FDA         Linha: 23

--- Final do Analisador Léxico. ---

```

Fonte: Elaborado pelos autores.

6.2 Teste 2 – NossoExemplo.txt

O segundo teste foi elaborado com o objetivo de validar o funcionamento do analisador em um programa mais simples, envolvendo atribuição, estrutura condicional e operadores aritméticos e relacionais.

```
valor <- 10.
limite <- 20.
se (valor < limite) entao
  valor <- valor + 5.
senao
  valor <- valor - 1.
fim_se.
escrever(valor).
```

A Figura 2 apresenta a saída produzida pelo analisador léxico para esse arquivo, confirmando que o analisador reconheceu corretamente todos os identificadores, números inteiros, palavras-chave, operadores e separadores.

Figura 2 – Saída do terminal para o Teste 2 (NossoExemplo.txt)

```
• viniciusnogueira@MacBook-Air-de-Vinicius AnalisadorLexico % flex analisador.l
gcc lex.yy.c -o analisador
./analisador
Início do Analisador Léxico.

Token: Identificador      | Valor: valor      | Linha: 1
Token: Operador de atribuicao | Valor: <-        | Linha: 1
Token: Numero inteiro     | Valor: 10         | Linha: 1
Token: Separador de comandos | Valor: .          | Linha: 1
Token: Identificador      | Valor: limite     | Linha: 2
Token: Operador de atribuicao | Valor: <-        | Linha: 2
Token: Numero inteiro     | Valor: 20         | Linha: 2
Token: Separador de comandos | Valor: .          | Linha: 2
Token: Palavra-chave      | Valor: se         | Linha: 3
Token: Abre bloco         | Valor: (          | Linha: 3
Token: Identificador      | Valor: valor      | Linha: 3
Token: Menor que          | Valor: <          | Linha: 3
Token: Identificador      | Valor: limite     | Linha: 3
Token: Fecha bloco        | Valor: )          | Linha: 3
Token: Palavra-chave      | Valor: entao      | Linha: 3
Token: Identificador      | Valor: valor      | Linha: 4
Token: Operador de atribuicao | Valor: <-        | Linha: 4
Token: Identificador      | Valor: valor      | Linha: 4
Token: Operacao de soma   | Valor: +          | Linha: 4
Token: Numero inteiro     | Valor: 5          | Linha: 4
Token: Separador de comandos | Valor: .          | Linha: 4
Token: Palavra-chave      | Valor: senao      | Linha: 5
Token: Identificador      | Valor: valor      | Linha: 6
Token: Operador de atribuicao | Valor: <-        | Linha: 6
Token: Identificador      | Valor: valor      | Linha: 6
Token: Operacao de subtracao | Valor: -          | Linha: 6
Token: Numero inteiro     | Valor: 1          | Linha: 6
Token: Separador de comandos | Valor: .          | Linha: 6
Token: Palavra-chave      | Valor: fim_se     | Linha: 7
Token: Separador de comandos | Valor: .          | Linha: 7
Token: Palavra-chave      | Valor: escrever   | Linha: 8
Token: Abre bloco         | Valor: (          | Linha: 8
Token: Identificador      | Valor: valor      | Linha: 8
Token: Fecha bloco        | Valor: )          | Linha: 8
Token: Separador de comandos | Valor: .          | Linha: 8
Token: Fim de arquivo     | Valor: FDA        | Linha: 8

Final do Analisador Léxico.
```

Fonte: Elaborado pelos autores.

6.3 Teste 3 – Reconhecimento de símbolos desconhecidos

O terceiro teste verifica o comportamento do analisador diante de caracteres que não pertencem ao alfabeto léxico da linguagem Domus ("@", "#", "\$" e "?"). Para isso, foi elaborado um arquivo de teste contendo esses símbolos intercalados com um programa válido, apresentado a seguir.

```
saldo <- 100.  
meta <- 150.  
@saldo <- saldo + 10.  
se (saldo < meta) entao  
    saldo <- saldo + 20.  
#meta <- meta - 5.  
senao  
    $saldo <- saldo - 10.  
fim_se.  
?meta <- meta + 1.  
escrever(saldo).
```

Conforme esperado, o analisador não interrompeu sua execução ao encontrar esses caracteres; em vez disso, cada um foi corretamente classificado como "Símbolo Desconhecido", sendo reportado com seu valor e a linha correspondente, e a análise prosseguiu normalmente até o fim do arquivo. Esse comportamento é resultado da regra genérica "." (qualquer caractere), posicionada por último na seção de regras do Flex, que captura qualquer caractere não reconhecido pelas demais regras.

Figura 3 – Saída do terminal para o Teste 3 (símbolos desconhecidos)

```

● viniciusnogueira@MacBook-Air-de-Vinicius AnalisadorLexico % flex analisador.l
gcc lex.yy.c -o analisador
./analisador
Início do Analisador Léxico.

Token: Identificador      Valor: saldo      Linha: 1
Token: Operador de atribuicao Valor: <-      Linha: 1
Token: Numero inteiro     Valor: 100      Linha: 1
Token: Separador de comandos Valor: .      Linha: 1
Token: Identificador      Valor: meta      Linha: 2
Token: Operador de atribuicao Valor: <-      Linha: 2
Token: Numero inteiro     Valor: 150      Linha: 2
Token: Separador de comandos Valor: .      Linha: 2
Token: Simbolo Desconhecido Valor: @      Linha: 3
Token: Identificador      Valor: saldo      Linha: 3
Token: Operador de atribuicao Valor: <-      Linha: 3
Token: Identificador      Valor: saldo      Linha: 3
Token: Operacao de soma   Valor: +      Linha: 3
Token: Numero inteiro     Valor: 10      Linha: 3
Token: Separador de comandos Valor: .      Linha: 3
Token: Palavra-chave      Valor: se      Linha: 4
Token: Abre bloco         Valor: (      Linha: 4
Token: Identificador      Valor: saldo      Linha: 4
Token: Menor que          Valor: <      Linha: 4
Token: Identificador      Valor: meta      Linha: 4
Token: Fecha bloco        Valor: )      Linha: 4
Token: Palavra-chave      Valor: entao     Linha: 4
Token: Identificador      Valor: saldo      Linha: 5
Token: Operador de atribuicao Valor: <-      Linha: 5
Token: Identificador      Valor: saldo      Linha: 5
Token: Operacao de soma   Valor: +      Linha: 5
Token: Numero inteiro     Valor: 20      Linha: 5
Token: Separador de comandos Valor: .      Linha: 5
Token: Simbolo Desconhecido Valor: #      Linha: 6
Token: Identificador      Valor: meta      Linha: 6
Token: Operador de atribuicao Valor: <-      Linha: 6
Token: Identificador      Valor: meta      Linha: 6
Token: Operacao de subtracao Valor: -      Linha: 6
Token: Numero inteiro     Valor: 5      Linha: 6
Token: Separador de comandos Valor: .      Linha: 6
Token: Palavra-chave      Valor: senao     Linha: 7
Token: Simbolo Desconhecido Valor: $      Linha: 8
Token: Identificador      Valor: saldo      Linha: 8
Token: Operador de atribuicao Valor: <-      Linha: 8
Token: Identificador      Valor: saldo      Linha: 8
Token: Operacao de subtracao Valor: -      Linha: 8
Token: Numero inteiro     Valor: 10      Linha: 8
Token: Separador de comandos Valor: .      Linha: 8
Token: Palavra-chave      Valor: fim_se    Linha: 9
Token: Separador de comandos Valor: .      Linha: 9
Token: Simbolo Desconhecido Valor: ?      Linha: 10
Token: Identificador      Valor: meta      Linha: 10
Token: Operador de atribuicao Valor: <-      Linha: 10
Token: Identificador      Valor: meta      Linha: 10
Token: Operacao de soma   Valor: +      Linha: 10
Token: Numero inteiro     Valor: 1      Linha: 10
Token: Separador de comandos Valor: .      Linha: 10
Token: Palavra-chave      Valor: escrever   Linha: 11
Token: Abre bloco         Valor: (      Linha: 11
Token: Identificador      Valor: saldo      Linha: 11
Token: Fecha bloco        Valor: )      Linha: 11
Token: Separador de comandos Valor: .      Linha: 11
Token: Fim de arquivo     Valor: FDA      Linha: 11

Final do Analisador Léxico.

```

Fonte: Elaborado pelos autores.

7. CONSIDERAÇÕES FINAIS

O desenvolvimento deste trabalho permitiu compreender, na prática, o funcionamento da fase de análise léxica de um compilador e a utilização de um gerador automático de analisadores léxicos. O Flex mostrou-se adequado para a definição dos padrões léxicos da linguagem Domus, exigindo atenção a aspectos como a ordem das regras e o tratamento de comentários em múltiplas linhas.

Os três testes realizados confirmaram que o analisador reconheceu corretamente as categorias de tokens definidas para a linguagem Domus, além de identificar símbolos desconhecidos sem interromper a execução do programa.

REFERÊNCIAS

GEEKSFORGEEKS. Flex (Fast Lexical Analyzer Generator). Disponível em: <https://www.geeksforgeeks.org/compiler-design/flex-fast-lexical-analyzer-generator/>.

Acesso em: 17 set. 2026.

AHO, A. V. et al. Compiladores: princípios, técnicas e ferramentas. 2. ed. São Paulo: Pearson Addison-Wesley, 2008. Disponível em:

https://archive.org/details/compiladores-principios-tecnicas-e-ferramentas_202405.

Acesso em: 17 set. 2026.